DATA IMPORTATION, CLEANING AND WRANGLING/MANIPULATION

1. DATA IMPORTATION

1.1. Install and Laod the Required Libararies

In [5]:

```
#!pip install pandas
#!pip install numpy
#!pip install matplotlib
#!pip install seaborn
```

1.2. Loading the Libraries

In [53]:

```
import pandas as pd                          ## For data impportation and handling data f
import numpy as np                           ## For handling numeric operations
import matplotlib.pyplot as plt              ## For data visualization
import seaborn as sn                         ## For data visualization

### Filter warnings
import warnings
warnings.filterwarnings("ignore")
```

1.3. Reading the Dataset into Jupyter Notebook

In [7]:

```
df = pd.read_csv("GSSsubset1.csv")
```

1.4. General Data Inspection

1.4.1. Check the Column Names

In [14]:

```
print(f"Columns: {df.columns.tolist()}")
```

```
Columns: ['id', 'sex', 'degree', 'income', 'marital', 'age', 'height', 'weight', 'hrswr
k']
```

1.4.2. Check the Shape of the Data

In [19]:

```
print(f"Shape: {df.shape}")
```

```
Shape: (3000, 9)
```

1.4.3. Check the Variable Typpes

In [20]:

```
df.info()
```

```
<class 'pandas.DataFrame'>
RangeIndex: 3000 entries, 0 to 2999
Data columns (total 9 columns):
 #   Column   Non-Null Count  Dtype
---  ------   --------------  -----
 0   id       3000 non-null   int64
 1   sex      3000 non-null   str
 2   degree   2942 non-null   str
 3   income   2767 non-null   float64
 4   marital  2934 non-null   str
 5   age      3000 non-null   int64
 6   height   3000 non-null   float64
 7   weight   2879 non-null   float64
 8   hrswrk   2933 non-null   float64
dtypes: float64(4), int64(2), str(3)
memory usage: 211.1 KB
```

1.4.4. Check the First Few Observations

In [10]:

```
df.head(20)
```

Out[10]:

	id	sex	degree	income	marital	age	height	weight	hrswrk
0	1	MALE	HIGH SCHOOL	26722.56	NEVER MARRIED	18	73.2	86.3	35.0
1	2	MALE	NaN	36544.91	MARRIED	35	69.9	95.3	47.0
2	3	FEMALE	HIGH SCHOOL	59587.14	DIVORCED	28	70.1	60.9	16.0
3	4	FEMALE	GRADUATE	NaN	MARRIED	43	61.9	76.0	44.0
4	5	MALE	HIGH SCHOOL	48899.47	MARRIED	48	65.8	58.9	44.0
5	6	FEMALE	HIGH SCHOOL	47790.38	MARRIED	43	60.7	68.4	46.0
6	7	FEMALE	BACHELOR	102903.23	MARRIED	42	66.5	66.7	51.0

	id	sex	degree	income	marital	age	height	weight	hrswrk
7	8	MALE	HIGH SCHOOL	36707.09	NEVER MARRIED	52	65.9	76.8	48.0
8	9	FEMALE	BACHELOR	48404.51	MARRIED	35	66.8	94.6	28.0
9	10	MALE	HIGH SCHOOL	29044.70	NEVER MARRIED	18	69.3	85.2	NaN
10	11	FEMALE	GRADUATE	419380.86	SEPARATED	58	62.4	64.8	31.0
11	12	MALE	JUNIOR COLLEGE	14995.63	WIDOWED	40	66.8	68.7	45.0
12	13	FEMALE	LT HIGH SCHOOL	22832.15	WIDOWED	52	61.3	61.0	25.0
13	14	MALE	LT HIGH SCHOOL	23183.92	NaN	50	71.6	100.4	55.0
14	15	FEMALE	GRADUATE	114886.28	MARRIED	40	64.1	75.5	42.0
15	16	FEMALE	BACHELOR	59232.59	MARRIED	51	62.9	72.9	20.0
16	17	MALE	BACHELOR	NaN	NEVER MARRIED	24	69.3	84.9	50.0
17	18	FEMALE	HIGH SCHOOL	37509.86	SEPARATED	49	61.5	58.8	31.0
18	19	FEMALE	LT HIGH SCHOOL	20512.23	WIDOWED	60	59.0	62.9	22.0
19	20	FEMALE	BACHELOR	84696.68	DIVORCED	63	59.7	65.5	42.0

1.4.5.View the last Few Obervations

In [12]:

```
df.tail(10)
```

Out[12]:

	id	sex	degree	income	marital	age	height	weight	hrswrk
2990	2991	FEMALE	BACHELOR	47934.48	MARRIED	42	64.6	62.9	36.0
2991	2992	MALE	HIGH SCHOOL	NaN	MARRIED	53	70.0	83.7	46.0
2992	2993	MALE	GRADUATE	308066.05	MARRIED	37	68.6	92.4	61.0
2993	2994	MALE	HIGH SCHOOL	NaN	MARRIED	58	66.5	86.3	35.0
2994	2995	FEMALE	HIGH SCHOOL	69153.23	DIVORCED	49	62.1	59.7	39.0
2995	2996	MALE	HIGH SCHOOL	42319.59	MARRIED	60	69.3	85.8	51.0
2996	2997	MALE	GRADUATE	175090.66	MARRIED	57	69.7	98.5	51.0
2997	2998	FEMALE	LT HIGH SCHOOL	12813.84	WIDOWED	51	62.2	62.8	42.0
2998	2999	FEMALE	GRADUATE	61419.36	NEVER MARRIED	18	61.2	60.4	48.0
2999	3000	FEMALE	HIGH SCHOOL	78015.07	MARRIED	33	66.3	72.8	8.0

1.5. General Descriptive Statistics

In [21]:

```
df.describe()
```

Out[21]:

	id	income	age	height	weight	hrswrk
count	3000.000000	2767.000000	3000.000000	3000.000000	2879.000000	2933.000000
mean	1500.500000	63871.815790	42.078000	67.400500	76.389232	39.778043
std	866.169729	51634.075898	13.314316	3.641297	14.584233	12.594723
min	1.000000	3720.040000	18.000000	58.100000	45.000000	0.000000
25%	750.750000	31989.025000	33.000000	64.600000	65.800000	34.000000
50%	1500.500000	49586.540000	42.000000	67.200000	75.700000	41.000000
75%	2250.250000	78986.595000	51.000000	70.100000	85.700000	48.000000
max	3000.000000	609741.180000	80.000000	78.600000	120.000000	79.000000

In [22]:

```
### Remove the id Column from the Summary Statistics
df.drop(columns = ["id"]).describe()
```

Out[22]:

	income	age	height	weight	hrswrk
count	2767.000000	3000.000000	3000.000000	2879.000000	2933.000000
mean	63871.815790	42.078000	67.400500	76.389232	39.778043
std	51634.075898	13.314316	3.641297	14.584233	12.594723
min	3720.040000	18.000000	58.100000	45.000000	0.000000
25%	31989.025000	33.000000	64.600000	65.800000	34.000000
50%	49586.540000	42.000000	67.200000	75.700000	41.000000
75%	78986.595000	51.000000	70.100000	85.700000	48.000000
max	609741.180000	80.000000	78.600000	120.000000	79.000000

1.6. General Frequency Tables

In [26]:

```
print("="*110)
print("Distribution of Gender in the General Social Survey Data")
print("="*110)

df["sex"].value_counts()
```

```
==============================================================================================================
Distribution of Gender in the General Social Survey Data
==============================================================================================================
```

Out[26]:

```
sex
FEMALE    1563
MALE      1437
Name: count, dtype: int64
```

In [27]:

```
print("="*110)
print("Distribution of Education Qualification in the General Social Survey Data")
print("="*110)

df["degree"].value_counts()
```

```
==============================================================================================================
Distribution of Education Qualification in the General Social Survey Data
==============================================================================================================
Out[27]:
degree
HIGH SCHOOL       1094
LT HIGH SCHOOL     583
BACHELOR           566
JUNIOR COLLEGE     465
GRADUATE           234
Name: count, dtype: int64
```

In [28]:

```
print("="*110)
print("Distribution of Marital Status in the General Social Survey Data")
print("="*110)

df["marital"].value_counts()
```

```
==============================================================================================================
Distribution of Marital Status in the General Social Survey Data
==============================================================================================================
Out[28]:
marital
MARRIED          1518
NEVER MARRIED     691
DIVORCED          382
WIDOWED           227
SEPARATED         116
Name: count, dtype: int64
```

2. DATA CLEANING

2.1. Handling Missing Values

In [29]:

```
df.head(20)
```

Out[29]:

	id	sex	degree	income	marital	age	height	weight	hrswrk
0	1	MALE	HIGH SCHOOL	26722.56	NEVER MARRIED	18	73.2	86.3	35.0
1	2	MALE	NaN	36544.91	MARRIED	35	69.9	95.3	47.0
2	3	FEMALE	HIGH SCHOOL	59587.14	DIVORCED	28	70.1	60.9	16.0
3	4	FEMALE	GRADUATE	NaN	MARRIED	43	61.9	76.0	44.0

	id	sex	degree	income	marital	age	height	weight	hrswrk
4	5	MALE	HIGH SCHOOL	48899.47	MARRIED	48	65.8	58.9	44.0
5	6	FEMALE	HIGH SCHOOL	47790.38	MARRIED	43	60.7	68.4	46.0
6	7	FEMALE	BACHELOR	102903.23	MARRIED	42	66.5	66.7	51.0
7	8	MALE	HIGH SCHOOL	36707.09	NEVER MARRIED	52	65.9	76.8	48.0
8	9	FEMALE	BACHELOR	48404.51	MARRIED	35	66.8	94.6	28.0
9	10	MALE	HIGH SCHOOL	29044.70	NEVER MARRIED	18	69.3	85.2	NaN
10	11	FEMALE	GRADUATE	419380.86	SEPARATED	58	62.4	64.8	31.0
11	12	MALE	JUNIOR COLLEGE	14995.63	WIDOWED	40	66.8	68.7	45.0
12	13	FEMALE	LT HIGH SCHOOL	22832.15	WIDOWED	52	61.3	61.0	25.0
13	14	MALE	LT HIGH SCHOOL	23183.92	NaN	50	71.6	100.4	55.0
14	15	FEMALE	GRADUATE	114886.28	MARRIED	40	64.1	75.5	42.0
15	16	FEMALE	BACHELOR	59232.59	MARRIED	51	62.9	72.9	20.0
16	17	MALE	BACHELOR	NaN	NEVER MARRIED	24	69.3	84.9	50.0
17	18	FEMALE	HIGH SCHOOL	37509.86	SEPARATED	49	61.5	58.8	31.0
18	19	FEMALE	LT HIGH SCHOOL	20512.23	WIDOWED	60	59.0	62.9	22.0
19	20	FEMALE	BACHELOR	84696.68	DIVORCED	63	59.7	65.5	42.0

2.1.1. Check the Presence of Missing Values

In [30]:

```
print("="*110)
print("Checking the Presence of Missing Values in the General Social Survey Data")
print("="*110)

print(df.isna().sum())
```

```
==============================================================================================================
Checking the Presence of Missing Values in the General Social Survey Data
==============================================================================================================
id            0
sex           0
degree       58
income      233
marital      66
age           0
height        0
weight      121
hrswrk       67
dtype: int64
```

In [32]:

```
print("="*110)
print("Checking the Percentage of Missing Values per Variable in the General Social Surv
print("="*110)
```

```
Missing_values = df.isna().mean().round(4)*100
Missing_values
```

```
================================================================================================
======================
Checking the Percentage of Missing Values per Variable in the General Social Survey Data
================================================================================================
======================
```

Out[32]:

```
id          0.00
sex         0.00
degree      1.93
income      7.77
marital     2.20
age         0.00
height      0.00
weight      4.03
hrswrk      2.23
dtype: float64
```

2.1.2. Methods of Handling Missing Values

- Complete cases Analysis- Commonly used approach but Risky
- Mean and/or Median Imputation - For symmetric and asymmetric data
- Mode Imputation - For Categorical and Character data

2.1.3. Complete Cases Analysis (Row Deletion)

In [34]:

```
df.info()
```

```
<class 'pandas.DataFrame'>
RangeIndex: 3000 entries, 0 to 2999
Data columns (total 9 columns):
 #   Column   Non-Null Count  Dtype
---  ------   --------------  -----
 0   id       3000 non-null   int64
 1   sex      3000 non-null   str
 2   degree   2942 non-null   str
 3   income   2767 non-null   float64
 4   marital  2934 non-null   str
 5   age      3000 non-null   int64
 6   height   3000 non-null   float64
 7   weight   2879 non-null   float64
 8   hrswrk   2933 non-null   float64
dtypes: float64(4), int64(2), str(3)
memory usage: 211.1 KB
```

In [36]:

```
df_drop_all = df.dropna()
print(df_drop_all.isna().sum())
```

```
id          0
sex         0
degree      0
income      0
marital     0
age         0
height      0
```

```
weight     0
hrswrk     0
dtype: int64
```

In [38]:

```
df_drop_all.info()
```

```
<class 'pandas.DataFrame'>
Index: 2503 entries, 0 to 2999
Data columns (total 9 columns):
 #   Column   Non-Null Count  Dtype
---  ------   --------------  -----
 0   id       2503 non-null   int64
 1   sex      2503 non-null   str
 2   degree   2503 non-null   str
 3   income   2503 non-null   float64
 4   marital  2503 non-null   str
 5   age      2503 non-null   int64
 6   height   2503 non-null   float64
 7   weight   2503 non-null   float64
 8   hrswrk   2503 non-null   float64
dtypes: float64(4), int64(2), str(3)
memory usage: 195.5 KB
```

2.1.4. Mean - Imputation (for symmetric data)

In [39]:

```
## Identify the Numeric Columns
df_mean = df.copy()
df_mean.head()
```

Out[39]:

	id	sex	degree	income	marital	age	height	weight	hrswrk
0	1	MALE	HIGH SCHOOL	26722.56	NEVER MARRIED	18	73.2	86.3	35.0
1	2	MALE	NaN	36544.91	MARRIED	35	69.9	95.3	47.0
2	3	FEMALE	HIGH SCHOOL	59587.14	DIVORCED	28	70.1	60.9	16.0
3	4	FEMALE	GRADUATE	NaN	MARRIED	43	61.9	76.0	44.0
4	5	MALE	HIGH SCHOOL	48899.47	MARRIED	48	65.8	58.9	44.0

In [40]:

```
df_mean.info()
```

```
<class 'pandas.DataFrame'>
RangeIndex: 3000 entries, 0 to 2999
Data columns (total 9 columns):
 #   Column   Non-Null Count  Dtype
---  ------   --------------  -----
 0   id       3000 non-null   int64
 1   sex      3000 non-null   str
 2   degree   2942 non-null   str
 3   income   2767 non-null   float64
 4   marital  2934 non-null   str
 5   age      3000 non-null   int64
 6   height   3000 non-null   float64
 7   weight   2879 non-null   float64
 8   hrswrk   2933 non-null   float64
```

```
dtypes: float64(4), int64(2), str(3)
memory usage: 211.1 KB
```

In [41]:

```
numeric_cols = df_mean.select_dtypes(include = "number").columns
df_mean[numeric_cols] = df_mean[numeric_cols].fillna(df_mean[numeric_cols].mean())
```

In [44]:

```
print(df.isna().sum())
```

```
id           0
sex          0
degree      58
income     233
marital     66
age          0
height       0
weight     121
hrswrk      67
dtype: int64
```

In [43]:

```
print(df_mean.isna().sum())
```

```
id          0
sex         0
degree     58
income      0
marital    66
age         0
height      0
weight      0
hrswrk      0
dtype: int64
```

2.1.5. Median Imputation for Asymmetric Data

In [45]:

```
df_median = df.copy()
df_median.head()
```

Out[45]:

	id	sex	degree	income	marital	age	height	weight	hrswrk
0	1	MALE	HIGH SCHOOL	26722.56	NEVER MARRIED	18	73.2	86.3	35.0
1	2	MALE	NaN	36544.91	MARRIED	35	69.9	95.3	47.0
2	3	FEMALE	HIGH SCHOOL	59587.14	DIVORCED	28	70.1	60.9	16.0
3	4	FEMALE	GRADUATE	NaN	MARRIED	43	61.9	76.0	44.0
4	5	MALE	HIGH SCHOOL	48899.47	MARRIED	48	65.8	58.9	44.0

In [46]:

```
numeric_cols = df_median.select_dtypes(include = "number").columns
df_median[numeric_cols] = df_median[numeric_cols].fillna(df_median[numeric_cols].median(
```

In [47]:

```
print(df_median.isna().sum())
```

```
id          0
sex         0
degree     58
income      0
marital    66
age         0
height      0
weight      0
hrswrk      0
dtype: int64
```

2.1.5. Mode Imputation for Categorical and Character Data

In [48]:

```
df["degree"].value_counts()
```

Out[48]:

```
degree
HIGH SCHOOL       1094
LT HIGH SCHOOL     583
BACHELOR           566
JUNIOR COLLEGE     465
GRADUATE           234
Name: count, dtype: int64
```

In [49]:

```
df["sex"].value_counts()
```

Out[49]:

```
sex
FEMALE    1563
MALE      1437
Name: count, dtype: int64
```

In [50]:

```
df["marital"].value_counts()
```

Out[50]:

```
marital
MARRIED          1518
NEVER MARRIED     691
DIVORCED          382
WIDOWED           227
SEPARATED         116
Name: count, dtype: int64
```

In [54]:

```
cat_cols = df.select_dtypes(include = ["object", "category"]).columns
for col in cat_cols:
    mode_val = df[col].mode()
    if not mode_val.empty:
        df[col]= df[col].fillna(mode_val[0])
```

In [55]:

```
df.isna().sum()
```

Out[55]:

```
id          0
sex         0
degree      0
income    233
```

```
marital      0
age          0
height       0
weight     121
hrswrk      67
dtype: int64
```

In [56]:

```
df["degree"].value_counts()
```

Out[56]:

```
degree
HIGH SCHOOL       1152
LT HIGH SCHOOL     583
BACHELOR           566
JUNIOR COLLEGE     465
GRADUATE           234
Name: count, dtype: int64
```

In [57]:

```
numeric_cols = df.select_dtypes(include = "number").columns
df[numeric_cols] = df[numeric_cols].fillna(df[numeric_cols].mean())
```

2.1.7. Check the Presence of Missing Values after Imputation

In [58]:

```
df.isna().sum()
```

Out[58]:

```
id         0
sex        0
degree     0
income     0
marital    0
age        0
height     0
weight     0
hrswrk     0
dtype: int64
```

2.1.8. Check the Data Structure

In [59]:

```
df.info()
```

```
<class 'pandas.DataFrame'>
RangeIndex: 3000 entries, 0 to 2999
Data columns (total 9 columns):
 #   Column   Non-Null Count  Dtype
---  ------   --------------  -----
 0   id       3000 non-null   int64
 1   sex      3000 non-null   str
 2   degree   3000 non-null   str
 3   income   3000 non-null   float64
 4   marital  3000 non-null   str
 5   age      3000 non-null   int64
 6   height   3000 non-null   float64
 7   weight   3000 non-null   float64
 8   hrswrk   3000 non-null   float64
```

```
dtypes: float64(4), int64(2), str(3)
memory usage: 211.1 KB
```

2.2. Handling the Duplicate Entries

2.2.1. Check the Duplicate Entries

In [62]:

```
dir(df)
```

Out[62]:

```
['T',
 '_AXIS_LEN',
 '_AXIS_ORDERS',
 '_AXIS_TO_AXIS_NUMBER',
 '_HANDLED_TYPES',
 '__abs__',
 '__add__',
 '__and__',
 '__annotations__',
 '__array__',
 '__array_priority__',
 '__array_ufunc__',
 '__arrow_c_stream__',
 '__bool__',
 '__class__',
 '__contains__',
 '__copy__',
 '__dataframe__',
 '__deepcopy__',
 '__delattr__',
 '__delitem__',
 '__dict__',
 '__dir__',
 '__divmod__',
 '__doc__',
 '__eq__',
 '__finalize__',
 '__floordiv__',
 '__format__',
 '__ge__',
 '__getattr__',
 '__getattribute__',
 '__getitem__',
 '__getstate__',
 '__gt__',
 '__hash__',
 '__iadd__',
 '__iand__',
 '__ifloordiv__',
 '__imod__',
 '__imul__',
 '__init__',
 '__init_subclass__',
 '__invert__',
 '__ior__',
 '__ipow__',
 '__isub__',
 '__iter__',
```

```
'__itruediv__',
'__ixor__',
'__le__',
'__len__',
'__lt__',
'__matmul__',
'__mod__',
'__module__',
'__mul__',
'__ne__',
'__neg__',
'__new__',
'__or__',
'__pandas_priority__',
'__pos__',
'__pow__',
'__radd__',
'__rand__',
'__rdivmod__',
'__reduce__',
'__reduce_ex__',
'__repr__',
'__rfloordiv__',
'__rmatmul__',
'__rmod__',
'__rmul__',
'__ror__',
'__round__',
'__rpow__',
'__rsub__',
'__rtruediv__',
'__rxor__',
'__setattr__',
'__setitem__',
'__setstate__',
'__sizeof__',
'__str__',
'__sub__',
'__subclasshook__',
'__truediv__',
'__weakref__',
'__xor__',
'_accessors',
'_accum_func',
'_agg_examples_doc',
'_agg_see_also_doc',
'_align_for_op',
'_align_frame',
'_align_series',
'_append_internal',
'_arith_method',
'_arith_method_with_reindex',
'_attrs',
'_box_col_values',
'_can_fast_transpose',
'_check_copy_deprecation',
'_check_inplace_and_allows_duplicate_labels',
'_check_label_or_level_ambiguity',
'_clip_with_one_bound',
```

```
'_clip_with_scalar',
'_cmp_method',
'_combine_frame',
'_consolidate',
'_consolidate_inplace',
'_construct_axes_dict',
'_construct_result',
'_constructor',
'_constructor_from_mgr',
'_constructor_sliced',
'_constructor_sliced_from_mgr',
'_dir_additions',
'_dir_deletions',
'_dispatch_frame_op',
'_drop_axis',
'_drop_labels_or_levels',
'_ensure_valid_index',
'_find_valid_index',
'_flags',
'_flex_arith_method',
'_flex_cmp_method',
'_from_arrays',
'_from_mgr',
'_get_agg_axis',
'_get_axis',
'_get_axis_name',
'_get_axis_number',
'_get_axis_resolvers',
'_get_block_manager_axis',
'_get_bool_data',
'_get_cleaned_column_resolvers',
'_get_column_array',
'_get_index_resolvers',
'_get_item',
'_get_label_or_level_values',
'_get_numeric_data',
'_get_value',
'_get_values_for_csv',
'_getitem_bool_array',
'_getitem_multilevel',
'_getitem_slice',
'_gotitem',
'_hidden_attrs',
'_indexed_same',
'_info_axis',
'_info_axis_name',
'_info_axis_number',
'_info_repr',
'_init_mgr',
'_inplace_method',
'_internal_names',
'_internal_names_set',
'_is_homogeneous_type',
'_is_label_or_level_reference',
'_is_label_reference',
'_is_level_reference',
'_is_mixed_type',
'_iset_item',
'_iset_item_mgr',
```

```
'_iset_not_inplace',
'_iter_column_arrays',
'_ixs',
'_logical_func',
'_logical_method',
'_maybe_align_series_as_frame',
'_metadata',
'_mgr',
'_min_count_stat_function',
'_module_source',
'_needs_reindex_multi',
'_pad_or_backfill',
'_reduce',
'_reduce_axis1',
'_reindex_axes',
'_reindex_multi',
'_reindex_with_indexers',
'_rename',
'_replace_columnwise',
'_repr_data_resource_',
'_repr_fits_horizontal_',
'_repr_fits_vertical_',
'_repr_html_',
'_repr_latex_',
'_reset_cache',
'_sanitize_column',
'_select_dtypes_indices',
'_series',
'_set_axis',
'_set_axis_name',
'_set_axis_nocheck',
'_set_item',
'_set_item_frame_value',
'_set_item_mgr',
'_set_value',
'_setitem_array',
'_setitem_frame',
'_setitem_slice',
'_shift_with_freq',
'_should_reindex_frame_op',
'_slice',
'_stat_function',
'_stat_function_ddof',
'_to_dict_of_blocks',
'_to_latex_via_styler',
'_typ',
'_update_inplace',
'_validate_dtype',
'_values',
'_where',
'abs',
'add',
'add_prefix',
'add_suffix',
'age',
'agg',
'aggregate',
'align',
'all',
```

```
'any',
'apply',
'asfreq',
'asof',
'assign',
'astype',
'at',
'at_time',
'attrs',
'axes',
'between_time',
'bfill',
'boxplot',
'clip',
'columns',
'combine',
'combine_first',
'compare',
'convert_dtypes',
'copy',
'corr',
'corrwith',
'count',
'cov',
'cummax',
'cummin',
'cumprod',
'cumsum',
'degree',
'describe',
'diff',
'div',
'divide',
'dot',
'drop',
'drop_duplicates',
'droplevel',
'dropna',
'dtypes',
'duplicated',
'empty',
'eq',
'equals',
'eval',
'ewm',
'expanding',
'explode',
'ffill',
'fillna',
'filter',
'first_valid_index',
'flags',
'floordiv',
'from_arrow',
'from_dict',
'from_records',
'ge',
'get',
'groupby',
```

```
'gt',
'head',
'height',
'hist',
'hrswrk',
'iat',
'id',
'idxmax',
'idxmin',
'iloc',
'income',
'index',
'infer_objects',
'info',
'insert',
'interpolate',
'isetitem',
'isin',
'isna',
'isnull',
'items',
'iterrows',
'itertuples',
'join',
'keys',
'kurt',
'kurtosis',
'last_valid_index',
'le',
'loc',
'lt',
'map',
'marital',
'mask',
'max',
'mean',
'median',
'melt',
'memory_usage',
'merge',
'min',
'mod',
'mode',
'mul',
'multiply',
'ndim',
'ne',
'nlargest',
'notna',
'notnull',
'nsmallest',
'nunique',
'pct_change',
'pipe',
'pivot',
'pivot_table',
'plot',
'pop',
'pow',
```

```
'prod',
'product',
'quantile',
'query',
'radd',
'rank',
'rdiv',
'reindex',
'reindex_like',
'rename',
'rename_axis',
'reorder_levels',
'replace',
'resample',
'reset_index',
'rfloordiv',
'rmod',
'rmul',
'rolling',
'round',
'rpow',
'rsub',
'rtruediv',
'sample',
'select_dtypes',
'sem',
'set_axis',
'set_flags',
'set_index',
'sex',
'shape',
'shift',
'size',
'skew',
'sort_index',
'sort_values',
'squeeze',
'stack',
'std',
'style',
'sub',
'subtract',
'sum',
'swaplevel',
'tail',
'take',
'to_clipboard',
'to_csv',
'to_dict',
'to_excel',
'to_feather',
'to_hdf',
'to_html',
'to_iceberg',
'to_json',
'to_latex',
'to_markdown',
'to_numpy',
'to_orc',
```

```
 'to_parquet',
 'to_period',
 'to_pickle',
 'to_records',
 'to_sql',
 'to_stata',
 'to_string',
 'to_timestamp',
 'to_xarray',
 'to_xml',
 'transform',
 'transpose',
 'truediv',
 'truncate',
 'tz_convert',
 'tz_localize',
 'unstack',
 'update',
 'value_counts',
 'values',
 'var',
 'weight',
 'where',
 'xs']
```

In [63]:

```
dup_rows = df.duplicated().sum()
print(dup_rows)
```

```
0
```

2.2.1. Drop the Duplicate Entries if any Exists

In [64]:

```
clean_df = df.drop_duplicates()
clean_df.info()
```

```
<class 'pandas.DataFrame'>
RangeIndex: 3000 entries, 0 to 2999
Data columns (total 9 columns):
 #   Column   Non-Null Count  Dtype  
---  ------   --------------  -----  
 0   id       3000 non-null   int64  
 1   sex      3000 non-null   str    
 2   degree   3000 non-null   str    
 3   income   3000 non-null   float64
 4   marital  3000 non-null   str    
 5   age      3000 non-null   int64  
 6   height   3000 non-null   float64
 7   weight   3000 non-null   float64
 8   hrswrk   3000 non-null   float64
dtypes: float64(4), int64(2), str(3)
memory usage: 211.1 KB
```

2.3. Handling Outliers

In [ ]:

```

```

In [ ]:

```

```